1. Propósito y Alcance del Sistema

1.1 Propósito

SICEN es un sistema web para el Ministerio de Educación Pública (MEP) que automatiza y organiza el levantamiento, registro, seguimiento y validación de la información estadística de los centros educativos públicos y privados del país, reduciendo inconsistencias, duplicidad de datos y dificultades de seguimiento.

1.2 Alcance

Dentro del alcance

- Aplicación web para la gestión de censos escolares, con cuatro actores: Jefatura DAE, Técnico DAE, Director de centro educativo y Supervisor.
- Los requerimientos funcionales RF-01 a RF-17, que incluyen:
    - Creación, configuración y aprobación de formularios censales, con opción de guardarlos como plantilla.
    - Definición de fechas de apertura y cierre de censos.
    - Registro de colaboradores y asignación de roles.
    - Llenado, guardado como borrador, envío, corrección y reenvío de censos.
    - Aprobación o rechazo de censos y registro de observaciones por parte del Técnico DAE.
    - Consulta de avance y generación de reportes por actor.
- Inicio de sesión con correo institucional con formato name@mep.go.cr.
- Uso de la información precargada de los centros educativos (nombre, código y oferta educativa autorizada).

Fuera del alcance

- Página pública o landing page.
- Proceso propio de confirmación con contraseña para registrar colaboradores, asignar roles o abrir y cerrar censos, ya que la plataforma Saber cuenta con sus propios protocolos de seguridad.
- Mantenimiento o actualización del listado de centros educativos, que es entregado por el MEP.
- Aplicación móvil nativa y módulos del MEP distintos a la gestión de censos.

- Actores adicionales a los cuatro definidos.

Supuestos y limitaciones

- El desarrollo se realiza dentro del periodo académico del curso (14 semanas) y se gestiona con Jira.
- La Jefatura DAE define y aprueba la estructura de los formularios antes de configurar un censo.
- La información de los centros educativos está precargada en el sistema.

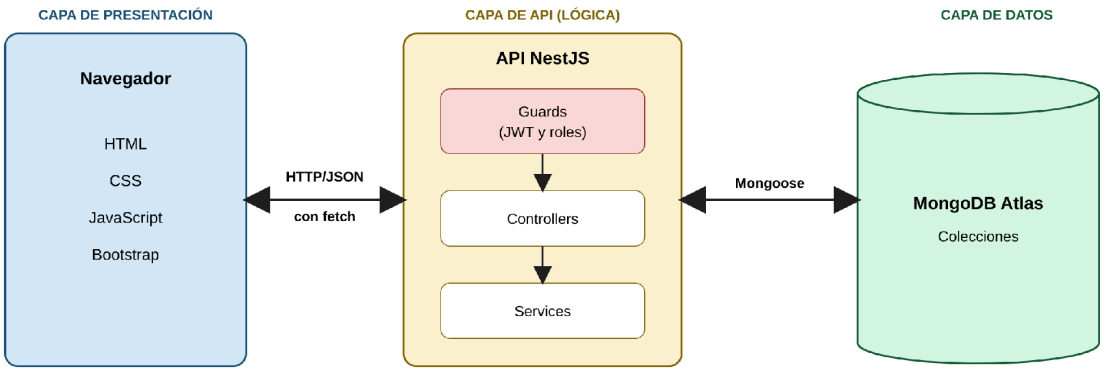
2. Arquitectura

SICEN sigue una arquitectura cliente-servidor en tres capas, con una API REST que separa la interfaz, la lógica de negocio y los datos.

2.1 Tabla de Capas

Capa	Responsabilidad	Tecnología
Presentación	Interfaz de usuario por rol, validaciones en el cliente y comunicación asíncrona con el servidor	HTML, CSS, JavaScript y Bootstrap
Lógica de negocio	API REST: autenticación, control de acceso por rol, reglas del censo (estados, fechas, aprobaciones) y generación de reportes	Node.js con NestJS
Datos	Almacenamiento persistente de usuarios, centros, formularios y censos	MongoDB Atlas

2.2 Diagrama por Capas



El sistema se organiza en tres capas:

- Presentación: el navegador, con HTML, CSS, JavaScript y Bootstrap. Muestra la interfaz y recoge las acciones del usuario.
- API: el servidor NestJS. Los guards validan el token JWT y el rol, los controllers reciben las solicitudes y los services contienen la lógica de negocio.
- Datos: MongoDB Atlas, que guarda las colecciones del sistema.

Comunicación entre capas:

- El navegador se comunica con la API mediante HTTP/JSON con fetch.
- La API se comunica con la base de datos mediante Mongoose.
- El navegador nunca accede directamente a la base de datos.

2.3 Flujo general

El navegador envía solicitudes HTTP a la API en formato JSON mediante fetch. La API valida la sesión y el rol del usuario, aplica las reglas de negocio, consulta o modifica MongoDB Atlas y devuelve la respuesta al cliente.

2.4 Patrones Arquitectónicos

Patrón	Dónde se aplica	Por qué se eligió
Cliente-servidor	El navegador (cliente) pide recursos y la API NestJS (servidor) responde.	Separa la interfaz de la lógica y de los datos. Permite que cada parte evolucione y se despliegue por separado. Las reglas del negocio y la seguridad quedan en el servidor, que es la parte de confianza.
Arquitectura en capas	Presentación, API y Datos, con comunicación solo entre capas vecinas.	Cada capa tiene una responsabilidad. Se puede cambiar una capa (por ejemplo, la interfaz) sin reescribir las demás, y el código queda más fácil de mantener y probar.
REST	La API expone recursos por URL y métodos HTTP (GET, POST, PUT, DELETE, PATCH) y responde en JSON.	Es un estándar conocido y sin estado. El cliente solo necesita fetch, y es sencillo de documentar y probar.
Arquitectura modular de NestJS con inyección de dependencias	Cada funcionalidad es un módulo con su controller, service y esquema. Las dependencias se inyectan por constructor.	Mantiene el código ordenado y con bajo acoplamiento. Los services se reutilizan entre módulos y se pueden reemplazar por simulados (mocks) en las pruebas.

Patrón	Dónde se aplica	Por qué se eligió
Guards para autenticación y roles	Un guard JWT verifica el token y otro verifica el rol antes de llegar al controller.	Centraliza la seguridad en un solo punto en lugar de repetirla en cada endpoint. Impide que una solicitud sin permiso llegue a la lógica de negocio.

2.5 Decisiones de diseño

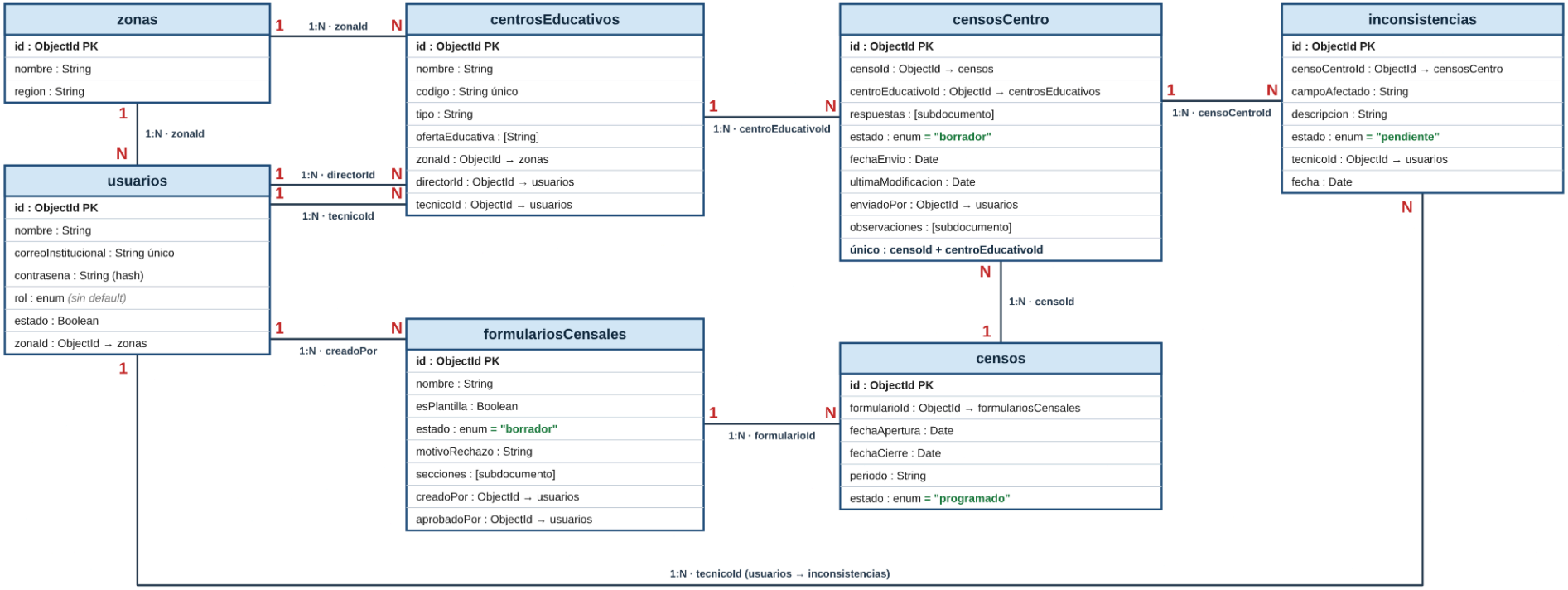
- Módulos por actor: el frontend y la API se organizan por actor y responsabilidad, para reducir duplicación de código.
- Convenciones: código en español, nombres de archivos y CSS en kebab-case, clases en PascalCase y variables y funciones en camelCase.
- Autenticación con JWT: el usuario inicia sesión y recibe un token firmado que envía en cada solicitud. Así la API no guarda sesiones en el servidor (es stateless), lo que encaja con REST. La seguridad adicional la aporta la plataforma Saber, por lo que el sistema no implementa confirmación con contraseña para acciones administrativas.
- Control de acceso con guards, en NestJS los guards deciden si una solicitud puede continuar. Tienen acceso al contexto de ejecución y al rol requerido por cada ruta, el guard de roles compara el rol del token con el exigido por la ruta.
- Acceso a datos con Mongoose: define esquemas y validaciones sobre MongoDB y mantiene la API desacoplada de la base de datos.

3. Base de Datos

Se usa MongoDB Atlas (NoSQL orientada a documentos). La estructura se organiza en las siguientes colecciones:

3.1 Diagrama de Colecciones

Diagrama de colecciones · SICEN (MongoDB Atlas)



Campo enum	Valores permitidos	Valor por defecto
usuarios.rol	Técnico DAE, Jefatura DAE, Director, Supervisor	sin default (se elige al registrar)
formulariosCensales.estado	borrador, pendiente, aprobado, rechazado	**borrador**
censos.estado	programado, abierto, cerrado	**programado**
censosCentro.estado	borrador, enviado, con_observaciones, aprobado, rechazado	**borrador**
inconsistencias.estado	pendiente, corregida	**pendiente**
formulariosCensales.secciones.campos.tipoDato	texto, numero, fecha, seleccion	**texto**

PK = clave primaria (_id). → = referencia a otra colección (ObjectId). 1 / N = cardinalidad. Verde = valor por defecto del enum.

Referencias a usuarios no dibujadas: formulariosCensales.aprobadoPor y censosCentro.enviadoPor (opcionales).

3.2 Detalle por colección

Todos los id son ObjectId generados automáticamente por MongoDB (_id).

3.2.1 usuarios

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
nombre	String	Sí	No	Mínimo 2 caracteres, permite tildes y ñ. Ej.: Juan Carlos Cruz Jiménez
correoInstitucional	String	Sí	Sí	Debe terminar en @mep.go.cr. Ej.: usuario@mep.go.cr
contrasena	String	Sí	No	Hash (Bcrypt)
rol	enum	Sí	No	Técnico DAE, Jefatura DAE, Director, Supervisor
estado	Boolean	Sí	No	true = activo, false = inactivo
zonaId	ObjectId	No	No	Ref: zonas. Solo para el Supervisor

Nota: el Director se liga a su centro desde centrosEducativos.directorId.

3.2.2 zonas

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
nombre	String	Sí	No	Nombre de la zona
region	String	Sí	No	Nombre de la región

3.2.3 centrosEducativos

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
nombre	String	Sí	No	CTP Guápiles
codigo	String	Sí	Sí	Alfanumérico. Ej.: CTP-123
tipo	String	Sí	No	Público o privado
ofertaEducativa	Array de String	Sí	No	["Académica", "Técnica"]
zonaId	ObjectId	Sí	No	Ref: zonas
directorId	ObjectId	No	No	Ref: usuarios
tecnicoId	ObjectId	No	No	Ref: usuarios

3.2.4 formulariosCensales

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
nombre	String	Sí	No	Nombre descriptivo, mínimo 5 caracteres
esPlantilla	Boolean	Sí	No	true o false
estado	enum	Sí	No	borrador, pendiente, aprobado, rechazado
motivoRechazo	String	No	No	Obligatorio si el estado es rechazado
secciones	Array de subdocumentos	Sí	No	Ver estructura abajo

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
creadoPor	ObjectId	Sí	No	Ref: usuarios
aprobadoPor	ObjectId	No	No	Ref: usuarios

Estructura de cada elemento de secciones:

Campo	Tipo	Obligatorio	Detalle
nombre	String	Sí	Nombre de la sección
campos	Array de subdocumentos	Sí	Lista de campos de la sección
campos.nombreCampo	String	Sí	Nombre del campo
campos.tipoDato	enum	Sí	texto, numero, fecha, seleccion
campos.obligatorio	Boolean	Sí	Indica si el campo es obligatorio al responder

3.2.5 censos

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
formularioId	ObjectId	Sí	No	Ref: formulariosCensales
fechaApertura	Date	Sí	No	Fecha y hora de apertura
fechaCierre	Date	Sí	No	Posterior a la fecha de apertura
periodo	String	Sí	No	Año o periodo del censo. Ej: 2026.
estado	enum	Sí	No	programado, abierto, cerrado

3.2.6 censosCentro

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
censoId	ObjectId	Sí	Compuesto	Ref: censos
centroEducativoId	ObjectId	Sí	Compuesto	Ref: centrosEducativos
respuestas	Array de subdocumentos	Sí	No	Ver estructura abajo
estado	enum	Sí	No	borrador, enviado, con_observaciones, aprobado, rechazado
fechaEnvio	Date	No	No	Se establece automáticamente al enviar
ultimaModificacion	Date	Sí	No	Fecha del último cambio
enviadoPor	ObjectId	No	No	Ref: usuarios
observaciones	Array de subdocumentos	No	No	Ver estructura abajo

Regla de unicidad compuesta: el par censoId + centroEducativoId es único (un centro tiene un solo registro por censo).

Estructura de cada elemento de respuestas:

Campo	Tipo	Obligatorio	Detalle
seccion	String	Sí	Nombre de la sección a la que pertenece el campo
nombreCampo	String	Sí	Debe coincidir con campos.nombreCampo del formulario

Campo	Tipo	Obligatorio	Detalle
valor	Mixed (texto, número, fecha o selección)	No en borrador, Sí al enviar	Se valida contra campos.tipoDato y campos.obligatorio

Estructura de cada elemento de observaciones:

Campo	Tipo	Obligatorio	Detalle
mensaje	String	Sí	Texto de la observación. Si el estado es rechazado, es el motivo.
autorId	ObjectId	Sí	Ref: usuarios (Técnico que la escribe)
fecha	Date	Sí	Se establece automáticamente al registrarla

3.2.7 inconsistencias (RF-08)

Campo	Tipo	Obligatorio	Único	Validación / ejemplo
id	ObjectId	Sí	Sí	Autogenerado
censoCentroId	ObjectId	Sí	No	Ref: censosCentro
campoAfectado	String	Sí	No	Nombre del campo con la inconsistencia
descripcion	String	Sí	No	Qué está mal y cómo corregirlo
estado	enum	Sí	No	pendiente, corregida
tecnicoId	ObjectId	Sí	No	Ref: usuarios
fecha	Date	Sí	No	Fecha en que se registró

Cabe recalcar que inconsistencias guarda los errores por campo, observaciones guarda mensajes generales y el motivo de rechazo.

3.3 Estados de censosCentro:

- con_observaciones: el Técnico encontró inconsistencias y el Director puede corregir y reenviar.

- rechazado: el Técnico lo devuelve con motivo obligatorio.
- aprobado: equivale a "Completado" en la interfaz.

3.4 Relaciones y cardinalidad

Relación	Cardinalidad	Campo que la implementa
zonas → centrosEducativos	1:N	centrosEducativos.zonaId
zonas → usuarios (Supervisor)	1:N	usuarios.zonaId
usuarios (Director) → centrosEducativos	1:N	centrosEducativos.directorId
usuarios (Técnico) → centrosEducativos	1:N	centrosEducativos.tecnicoId
usuarios → formulariosCensales (creador)	1:N	formulariosCensales.creadoPor
formulariosCensales → censos	1:N	censos.formularioId
censos → censosCentro	1:N	censosCentro.censoId
centrosEducativos → censosCentro	1:N	censosCentro.centroEducativoId
censosCentro → inconsistencias	1:N	inconsistencias.censoCentroId
usuarios (Técnico) → inconsistencias	1:N	inconsistencias.tecnicoId

- Un centro pertenece a una zona, tiene un Director y tiene un Técnico DAE asignado.
- Un censo usa un formulario censal.
- Cada centro genera un censosCentro por censo, que contiene las respuestas y las observaciones del Técnico.
- Los reportes y el avance por nivel nacional, zona o centro no tienen colección propia, se calculan con consultas de agregación sobre censosCentro.

No existe una colección de notificaciones. Cuando el Técnico aprueba, rechaza o devuelve con observaciones, cambia el estado del registro y el Director lo ve en su pantalla.

4. Estructura del Sistema

4.1 Carpeta frontend

Contiene todos los elementos necesarios para el funcionamiento del frontend, la carpeta assets para guardar imágenes, videos o audio, que se utilice en las páginas, la carpeta de css incluye todos los archivos de estilo en cascada necesarios para la correcta visualización de las páginas, incluyendo un css general para no escribir el mismo código en varios archivos, la carpeta js contiene todos los archivos de tipo JavaScript necesarios para la interactividad y validaciones con las páginas web, los archivos tipo HTML (estructura de la página) se encuentran en la raíz de la carpeta frontend.

4.2 Carpeta backend

Contiene el código de la API REST. Todo se organiza en la carpeta src, con una subcarpeta por módulo: una por colección (usuarios, zonas, centros-educativos, formularios-censales, censos, censos-centro, inconsistencias) y una por funcionalidad transversal (auth y reportes). Cada módulo agrupa su controller (rutas y endpoints), su service (lógica de negocio), su schema (esquema Mongoose de la colección) y la carpeta dto (validación de los datos de entrada). Los guards y decoradores para la autenticación y los roles están en auth. app.module.ts es el módulo raíz y main.ts arranca la aplicación.

4.3 Carpeta docs

Contiene los archivos relacionados a la documentación del proyecto, incluyendo la matriz de trazabilidad, la especificación de requisitos de software, la especificación de diseño de software, los mapas de sitio del sistema, el diagrama de casos de uso, y los prototipos asociados a cada requerimiento.

4.4 Árbol de carpetas del backend

```
backend/
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   ├── auth/
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── dto/
│   │   ├── guards/
│   │   └── decorators/
│   ├── usuarios/
│   ├── zonas/
│   ├── centros-educativos/
│   ├── formularios-censales/
│   ├── censos/
│   ├── censos-centro/
│   ├── inconsistencias/
│   ├── reportes/
│   └── common/
├── test/
├── .env
└── package.json, tsconfig.json, nest-cli.json
```

5. API REST

Requerimiento Asociado	**Método**	**Ruta**	**Descripción**
RF-01	POST	/api/formularios	Crea un formulario censal (nombre, secciones, campos, esPlantilla) en estado borrador.
RF-01	GET	/api/formularios	Lista formularios; filtros ?estado= y ?esPlantilla=true para las plantillas.
RF-02	GET	/api/formularios/pendientes	Lista de formularios pendientes de aprobación.
RF-01	GET	/api/formularios/:id	Devuelve un formulario con sus secciones y campos.
RF-01	PUT	/api/formularios/:id	Actualiza la estructura (agregar o borrar campos y secciones).
RF-01	POST	/api/formularios/:id/validar	Botón “Continuar”: verifica que estén

			definidos los campos obligatorios (criterio 3).
RF-02	PATCH	/api/formularios/:id/aprobar	Pasa el formulario a aprobado, guarda aprobadoPor y habilita el censo para los centros.
RF-02	PATCH	/api/formularios/:id/rechazar	Rechaza el formulario; el body lleva el motivo obligatorio.
RF-03	POST	/api/censos	Crea un censo con un formulario aprobado y sus fechas de apertura y cierre.
RF-03	GET	/api/censos	Lista censos con sus fechas (alimenta el calendario).
RF-03	PUT	/api/censos/:id/fechas	Modifica apertura y cierre.
RF-03	GET	/api/censos/proximos-a-cerrar	Censos que cierran en 3 días o menos, para mostrar el aviso.
RF-04	POST	/api/usuarios	Registra un colaborador (nombre, correo name@mep.go.cr, rol: Técnico DAE o Supervisor).
RF-04	GET	/api/usuarios	Lista colaboradores registrados.
RF-04	PATCH	/api/usuarios/:id/rol	Cambia el rol; rechaza si el :id es el del usuario que hace la petición.
RF-05	GET	/api/censos/estado	Cantidad de censos por estado (borrador, enviado, con_observaciones, aprobado, rechazado); filtros ?region= y ?centro=
RF-05	GET	/api/censos/:id/centros	Estado del censo por cada centro educativo (censosCentro).
RF-05	GET	/api/zonas	Datos para llenar los filtros por región.

RF-05	GET	/api/centros-educativos	Datos para llenar los filtros por centro.
RF-06	GET	/api/reportes/estadisticos	Datos estadísticos con ?periodos=N; indica si faltan datos o están completos.
RF-06	GET	/api/reportes/estadisticos/exportar	Exporta el reporte con ?formato=pdf o ?formato=xlsx.
RF-07	GET	/api/tecnico/censos	Censos de los centros asignados al Técnico con su estado; filtro ?estado=.
RF-07	PATCH	/api/censos-centro/:id/aprobar	Pasa el censo a “aprobado” solo si el centro está asignado al Técnico y no hay inconsistencias pendientes (RF-08, criterio 3).
RF-07	PATCH	/api/censos-centro/:id/rechazar	Rechaza el censo con motivo obligatorio. El estado pasa a rechazado y el Director lo ve en su pantalla.
RF-08	POST	/api/censos-centro/:id/inconsistencias	Registra una inconsistencia (descripción, campo afectado) en estado pendiente.
RF-08	GET	/api/censos-centro/:id/inconsistencias	Historial de inconsistencias del censo con su estado (pendiente o corregida).
RF-08	PATCH	/api/inconsistencias/:id	Cambia el estado a corregida.
RF-08	POST	/api/censos-centro/:id/inconsistencias/devolver-con-observaciones	Devuelve las inconsistencias al Director para que las corrija antes de la aprobación.
RF-09	GET	/api/reportes/avance	Reporte de avance de los centros asignados; con ?centroId= es de un solo centro. Incluye estado y observaciones, y

			responde 403 si el centro no es suyo.
RF-10	GET	/api/centros-educativos/mi-centro	Datos precargados del centro del Director (nombre, código, oferta). Es solo lectura: no hay POST ni PUT para esos campos.
RF-11	GET	/api/censos/vigente	Formulario habilitado del periodo vigente, ya con los datos precargados.
RF-11	POST	/api/censos-centro/:id/enviar	Envío definitivo. Rechaza si el periodo está inactivo. El estado pasa a enviado y el Técnico lo ve en su lista de censos.
RF-12	PUT	/api/censos-centro/:id/borrador	Guarda el borrador sin exigir todos los campos; rechaza si el periodo está inactivo.
RF-13	GET	/api/censos-centro/mis-censos	Lista de censos enviados con su estado, incluyendo periodos anteriores.
RF-12	GET	/api/censos-centro/:id	Recupera el censo (borrador incluido) para retomarlo en otra sesión.
RF-13	GET	/api/censos-centro/:id/observaciones	Detalle de las observaciones y el motivo cuando el estado es “con_observaciones”.
RF-14	PUT	/api/censos-centro/:id/corregir	Edita un censo con observaciones; solo en periodo activo.
RF-14	POST	/api/censos-centro/:id/reenviar	Reenvía el censo corregido y reinicia la revisión. El estado vuelve a enviado y el Técnico lo ve en su lista.
RF-15	GET	/api/supervisor/centros	Centros asignados al Supervisor (sirve para el filtro).
RF-15	GET	/api/supervisor/avance	Estado de los censos de sus centros; ?centroId=

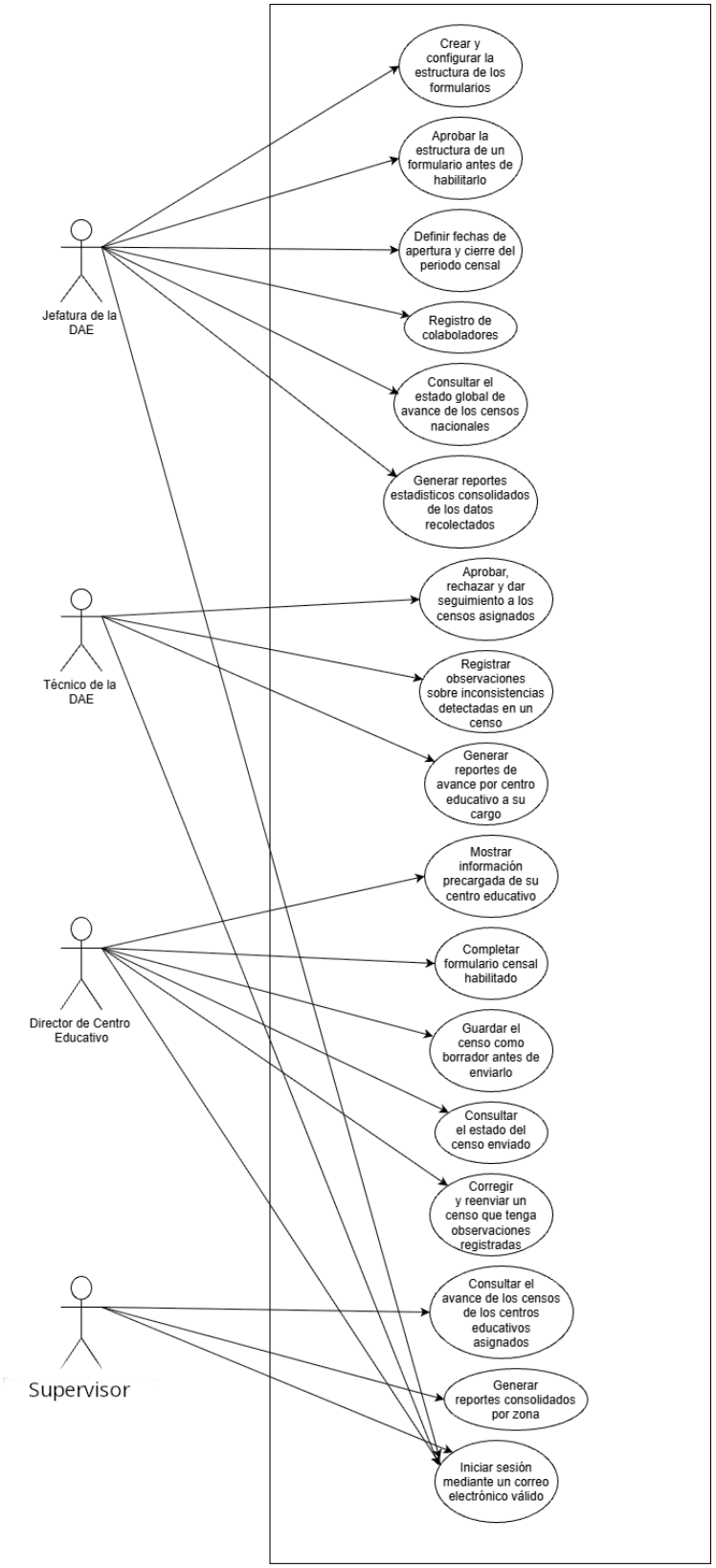
			filtra uno y responde 403 si no es suyo.
RF-16	GET	/api/reportes/zona	Reporte consolidado de la zona asignada; 403 si pide otra zona.
RF-16	GET	/api/reportes/zona/exportar	Exporta ese reporte con ?formato=pdf o ?formato=xlsx.
RF-17	POST	/api/auth/login	Recibe correo y contraseña. Valida el formato @mep.go.cr (400 si es inválido), verifica credenciales (401 si son incorrectas) y que el usuario esté activo (403 si no). Devuelve el token y el rol.
RF-17	POST	/api/auth/logout	Cierra la sesión (el “Cerrar sesión” del menú lateral).
RF-17	GET	/api/auth/me	Devuelve el usuario y rol de la sesión actual, para que el frontend sepa qué menú mostrar al cargar.

6. Tecnologías Utilizadas

Capa	Tecnología	Versión	Propósito
Frontend	HTML	5	Estructura de las páginas
Frontend	CSS	3	Estilos personalizados
Frontend	JavaScript	ES6+	Interactividad, validaciones y llamadas con fetch
Frontend	Bootstrap	5	Componentes visuales y diseño responsivo
Backend	Node.js	22.23.1	Entorno de ejecución del servidor
Backend	NestJS	12.2.0	Framework de la API REST
Backend	TypeScript	6.0.3	JavaScript tipado para el backend
Backend	class-validator / class-transformer	0.15.1 / 0.5.1	Validar y transformar los datos de entrada (DTO)
Backend	@nestjs/config	12.0.1	Variables de entorno (conexión a Atlas, clave del JWT)
Autenticación	bcrypt	6.0.0	Hash de contraseñas
Autenticación	@nestjs/jwt	12.0.4	Generar y verificar el token JWT
Exportación	pdfmake	0.3.14	Reportes en PDF
Exportación	exceljs	4.4.0	Reportes en Excel (.xlsx)
Base de datos	MongoDB Atlas	8.0.36	Almacenamiento en la nube
Base de datos	Mongoose / @nestjs/mongoose	9.11.2 / 12.0.0	Esquemas y acceso a MongoDB
Pruebas	Jest / Supertest	30.5.2 / 7.3.1	Pruebas unitarias y de la API (opcional)
Control de versiones	Git	2.56.0	Control de versiones
Gestión	Jira	Cloud	Requerimientos e historias de usuario

Modelado, Guía de Estilos, Usabilidad y Diseño de Navegación

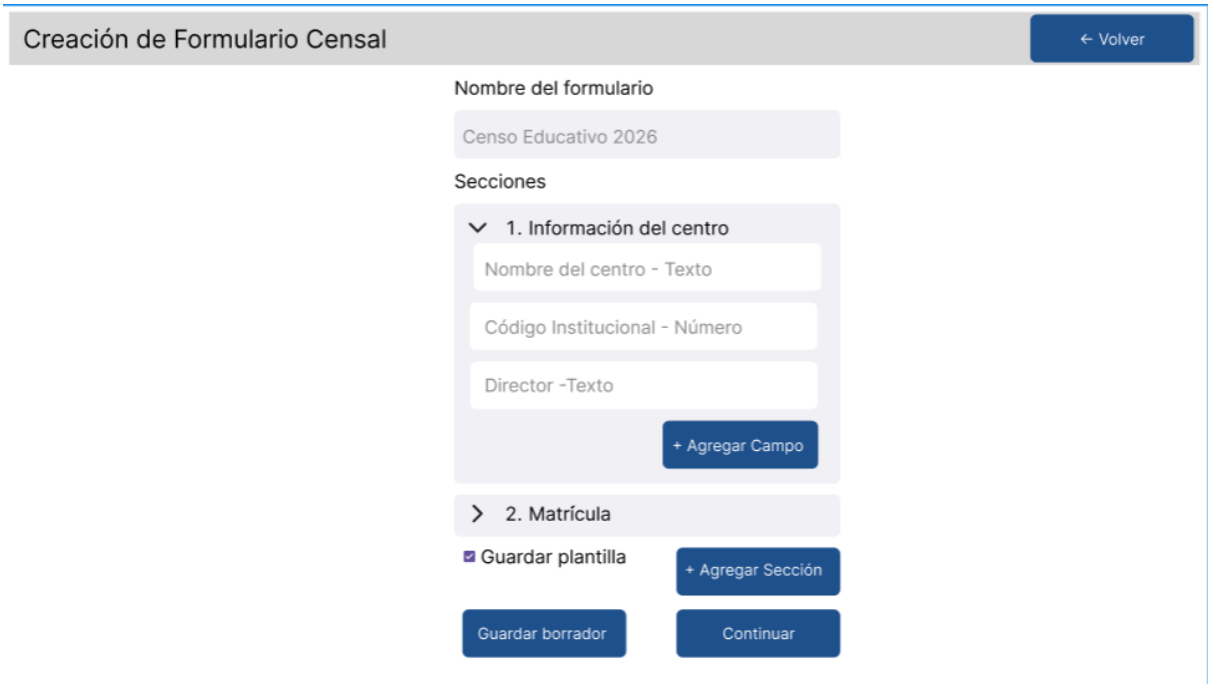
7.1 Modelado del Sistema



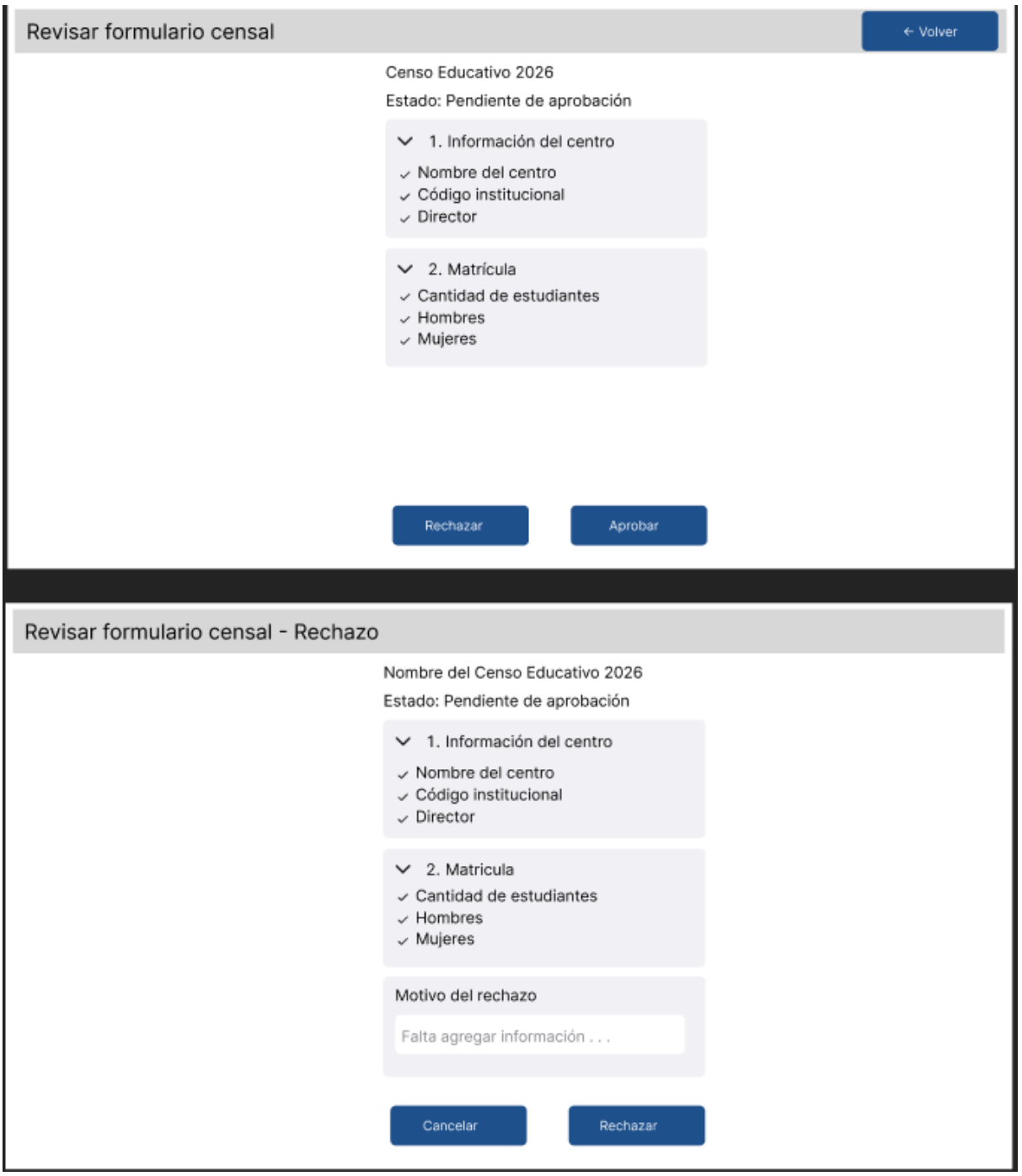
7.2 Interfaz de Usuario (UI/UX)

7.2.1 Prototipos

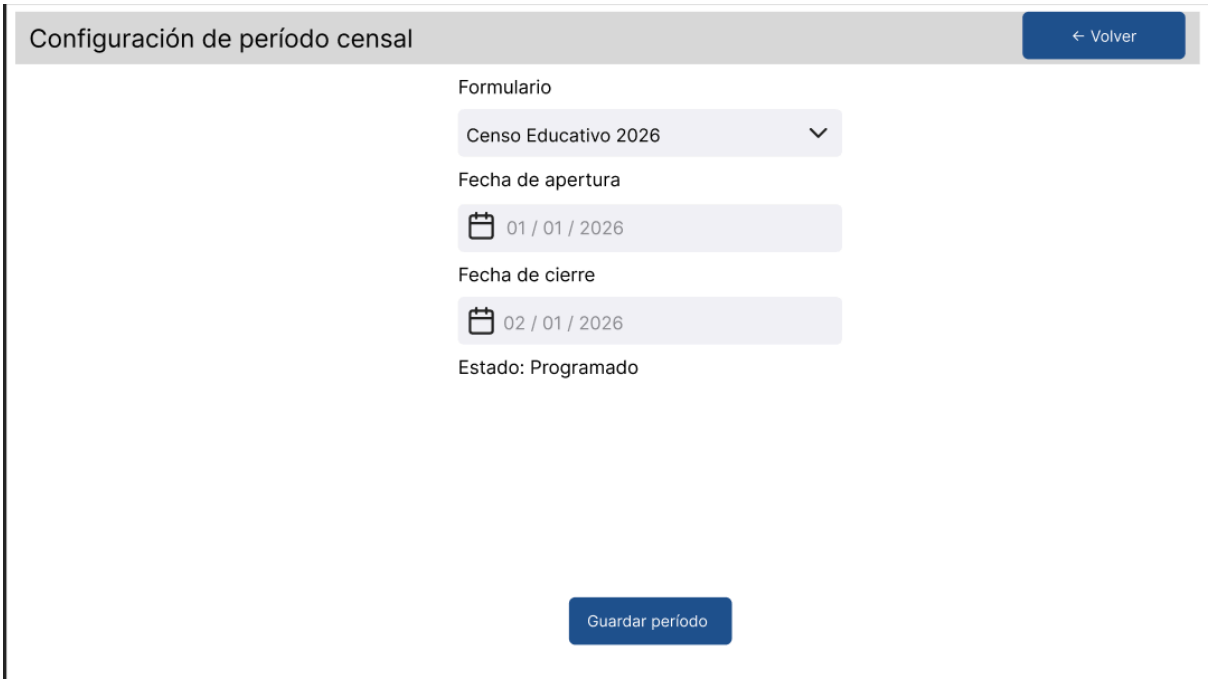
7.2.1.1 RF-01



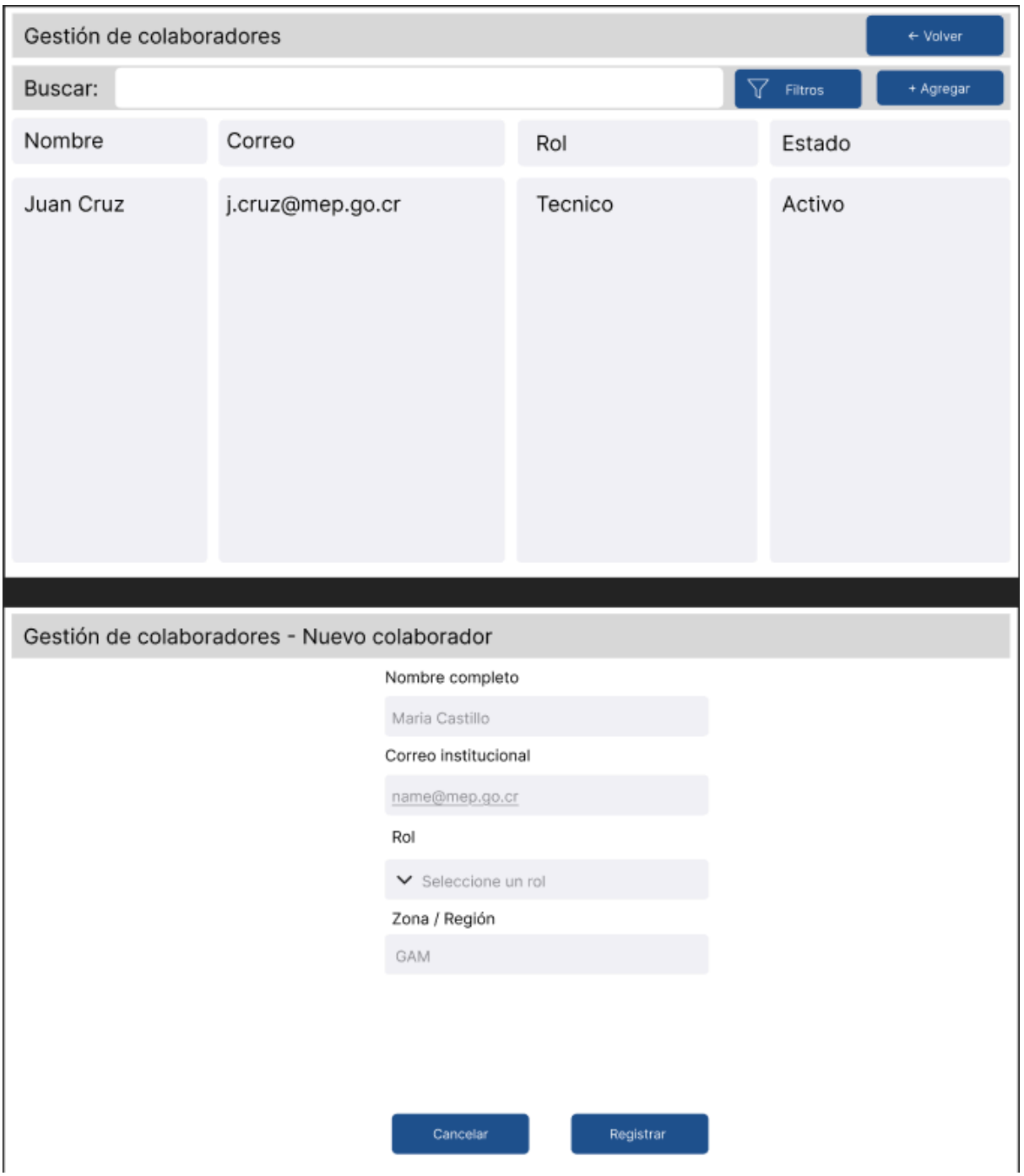
7.2.1.2 RF-02


Revisar formulario censal
← Volver
Censo Educativo 2026
Estado: Pendiente de aprobación
1. Información del centro
Nombre del centro
Código institucional
Director
2. Matrícula
Cantidad de estudiantes
Hombres
Mujeres
Rechazar
Aprobar
Revisar formulario censal - Rechazo
Nombre del Censo Educativo 2026
Estado: Pendiente de aprobación
1. Información del centro
Nombre del centro
Código institucional
Director
2. Matricula
Cantidad de estudiantes
Hombres
Mujeres
Motivo del rechazo
Falta agregar información . . .
Cancelar
Rechazar

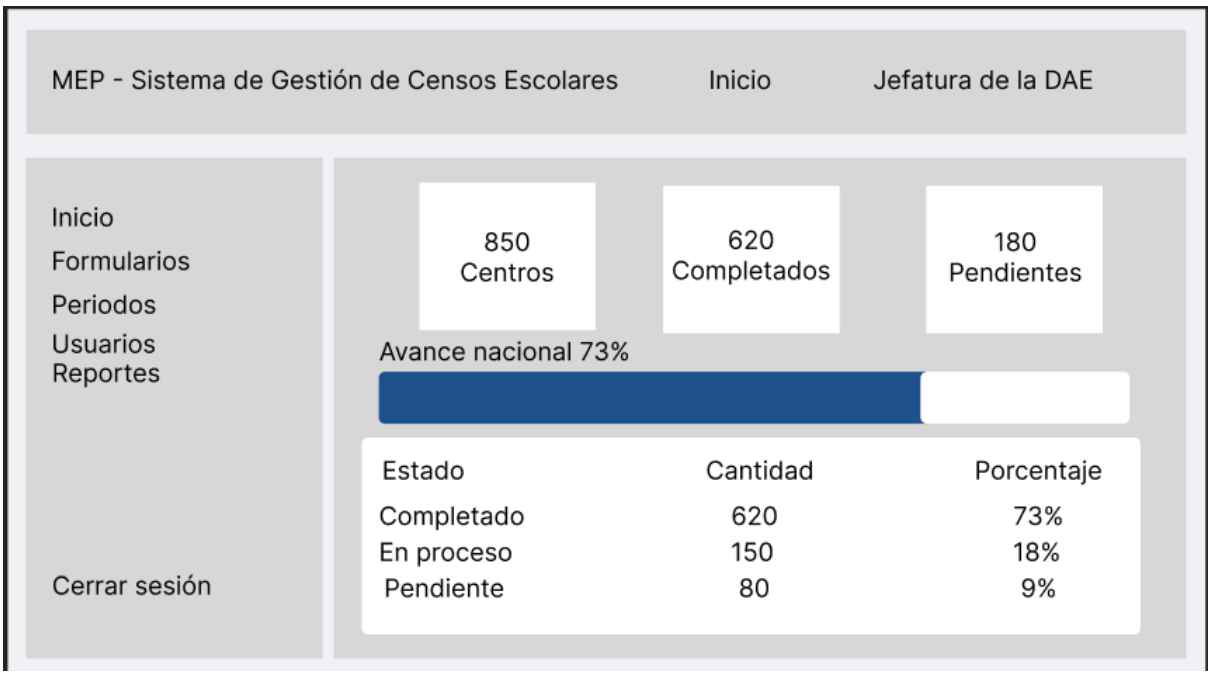
7.2.1.3 RF-03


Configuración de período censal
← Volver
Formulario
Censo Educativo 2026
Fecha de apertura
01 / 01 / 2026
Fecha de cierre
02 / 01 / 2026
Estado: Programado
Guardar período

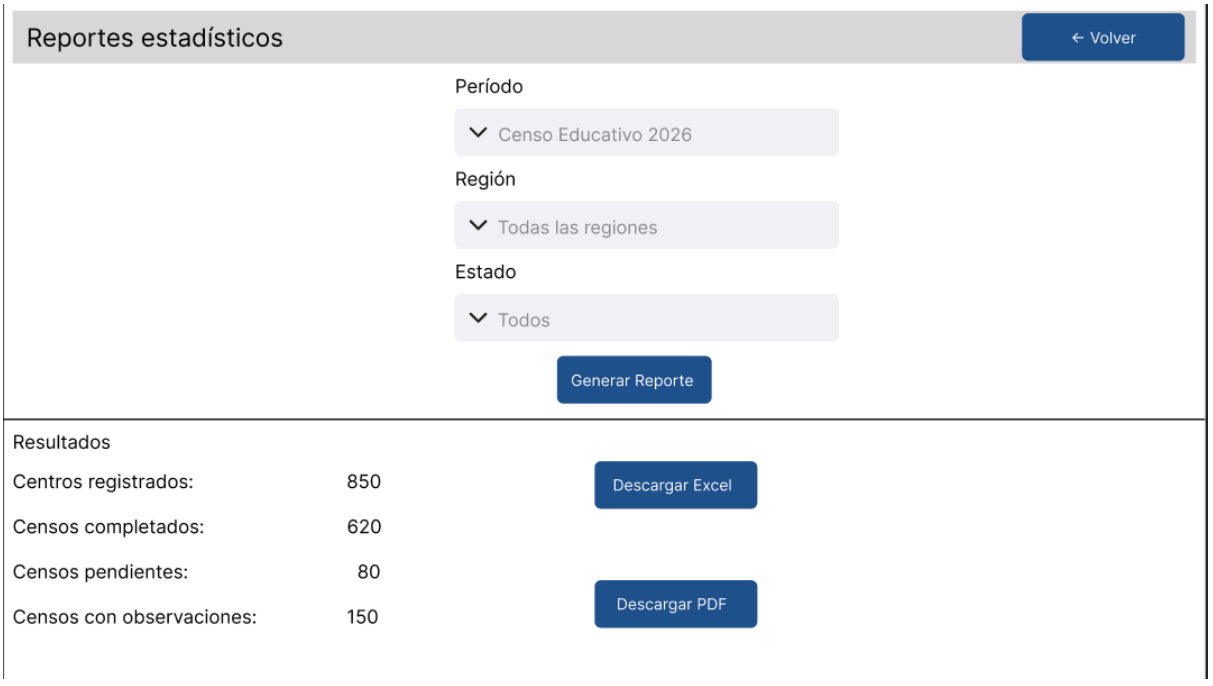
7.2.1.4 RF-04


Gestión de colaboradores
← Volver
Buscar:
Filtros
+ Agregar
Nombre
Correo
Rol
Estado
Juan Cruz
j.cruz@mep.go.cr
Tecnico
Activo
Gestión de colaboradores - Nuevo colaborador
Nombre completo
Maria Castillo
Correo institucional
name@mep.go.cr
Rol
Seleccione un rol
Zona / Región
GAM
Cancelar
Registrar

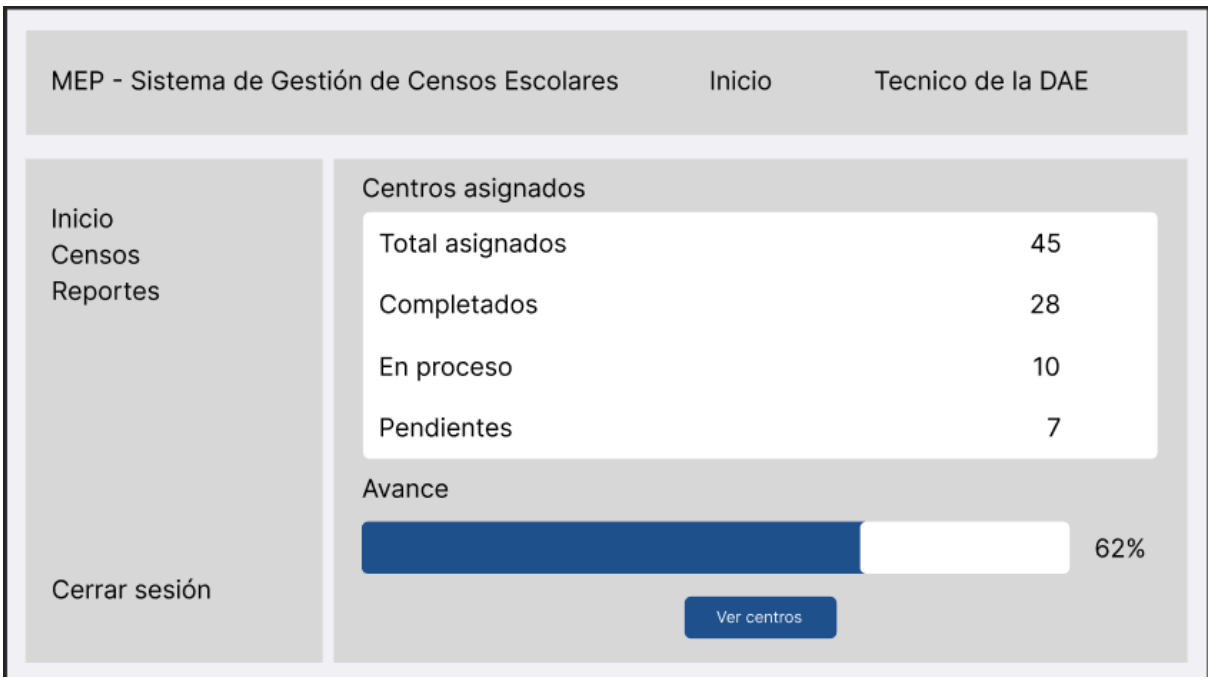
7.2.1.5 RF-05

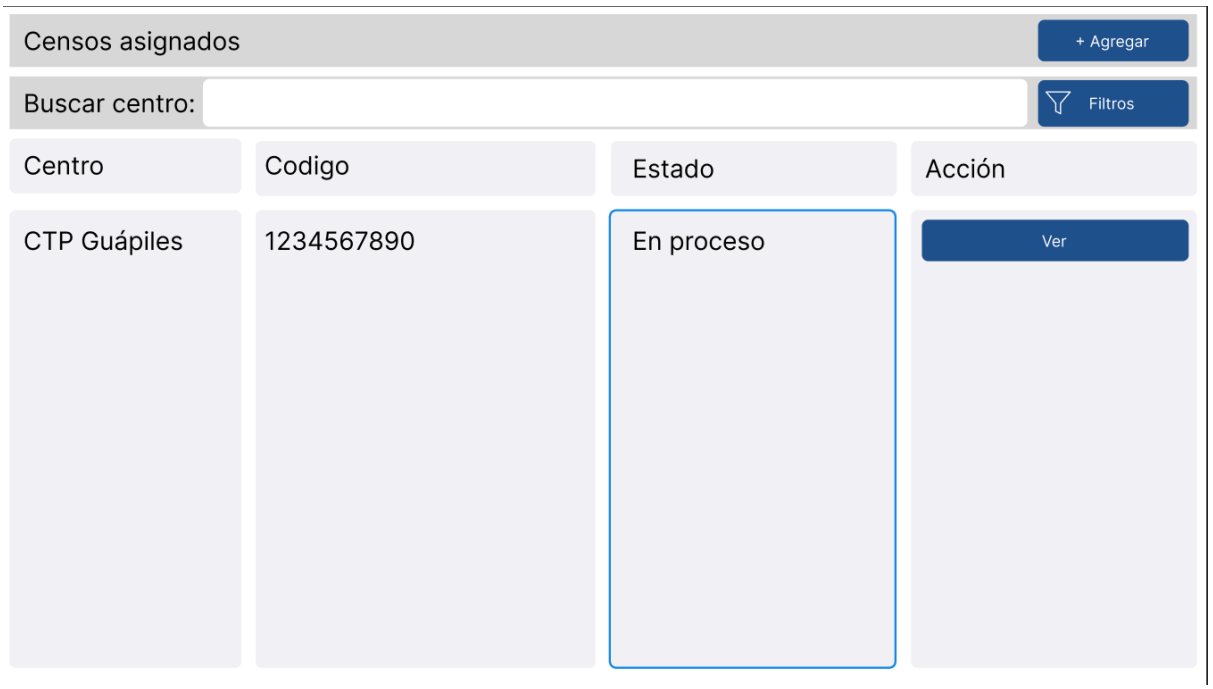


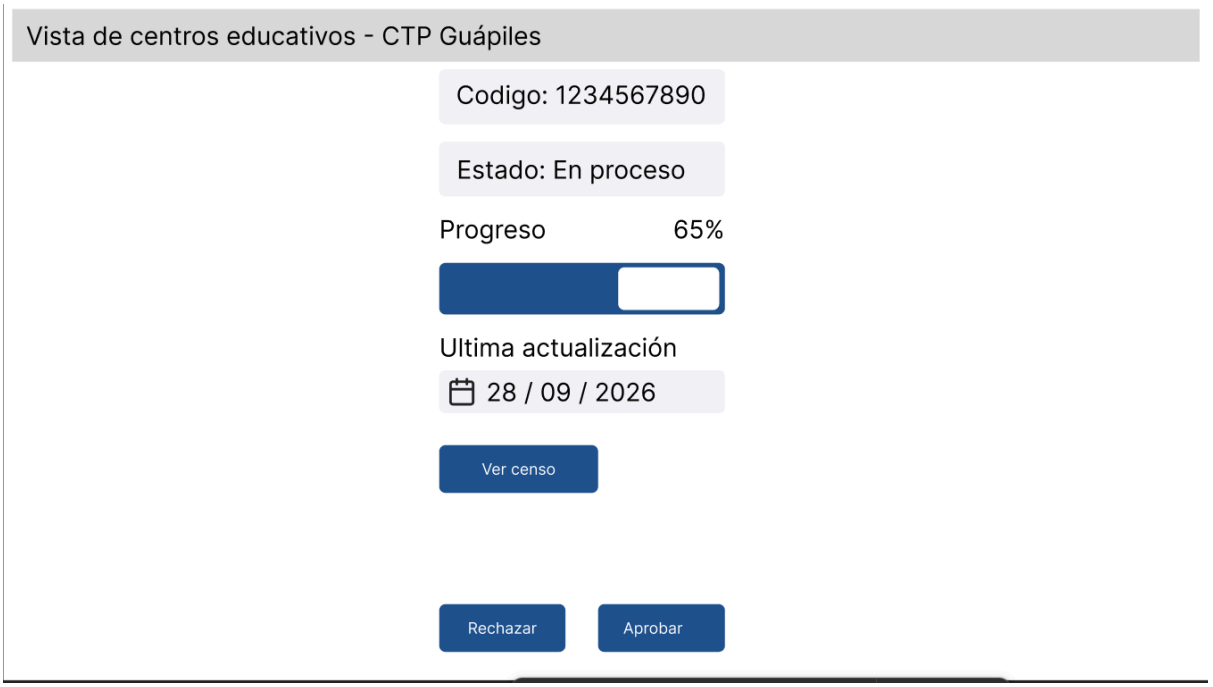
7.2.1.6 RF-06



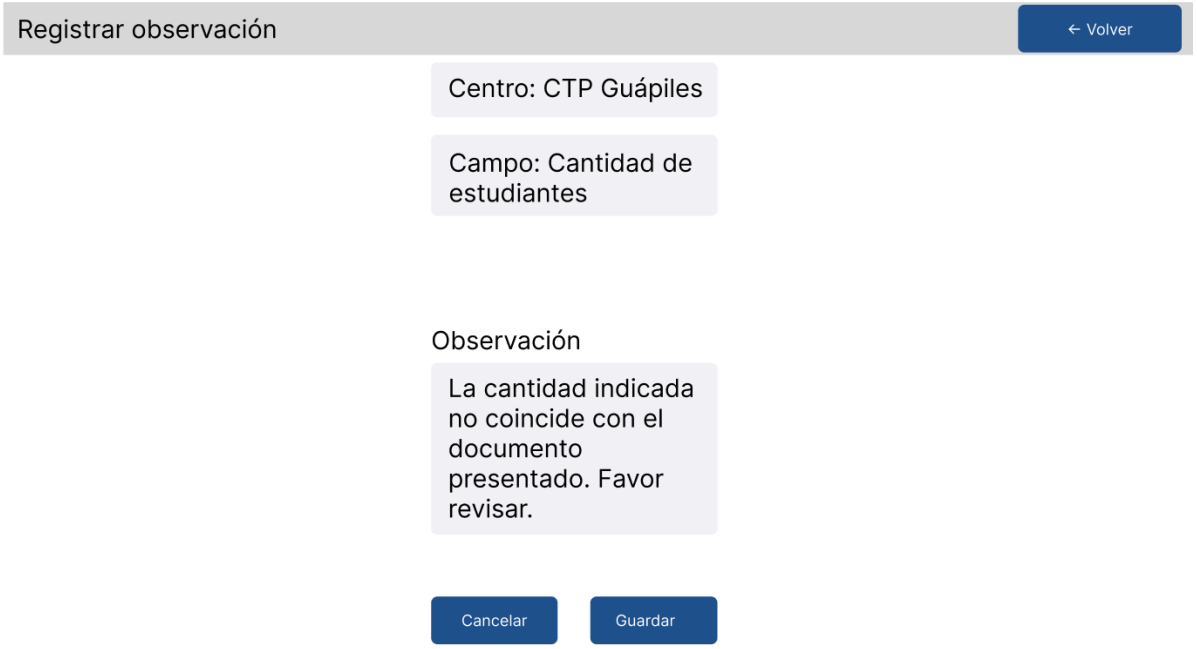
7.2.1.7 RF-07


MEP - Sistema de Gestión de Censos Escolares
Inicio
Tecnico de la DAE
Inicio
Censos
Reportes
Cerrar sesión
Centros asignados
Total asignados 45
Completados 28
En proceso 10
Pendientes 7
Avance
62%
Ver centros

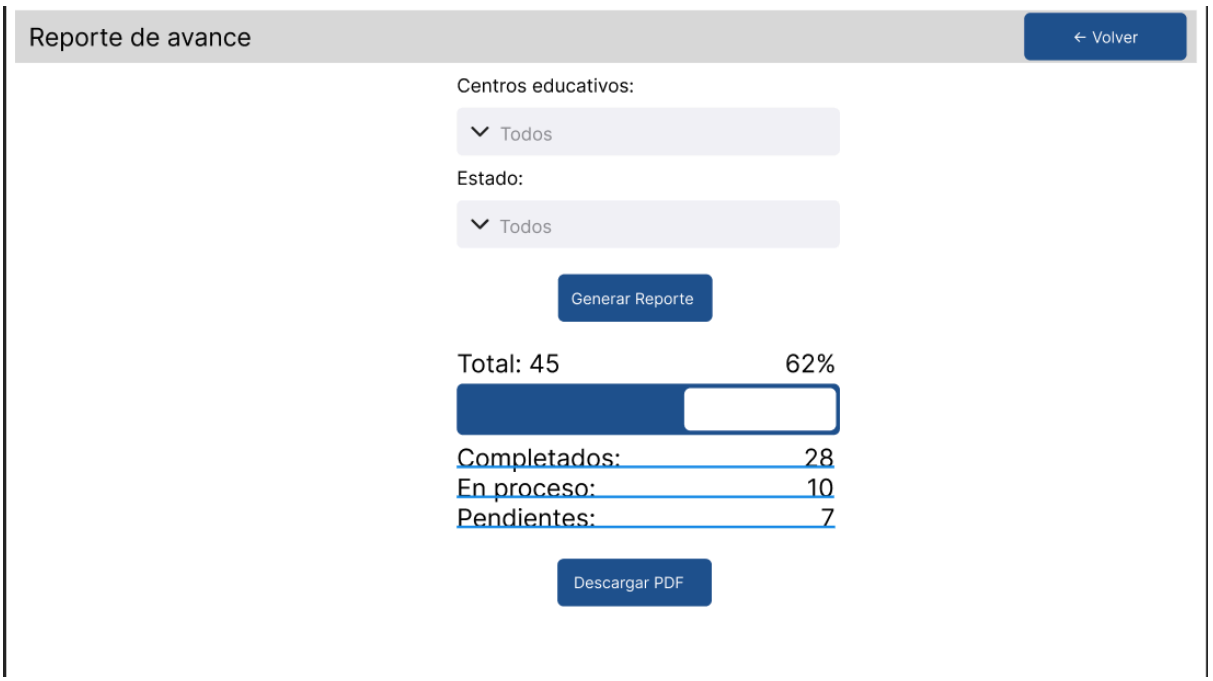

Censos asignados
+ Agregar
Buscar centro:
Filtros
Centro
Codigo
Estado
Acción
CTP Guápiles
1234567890
En proceso
Ver



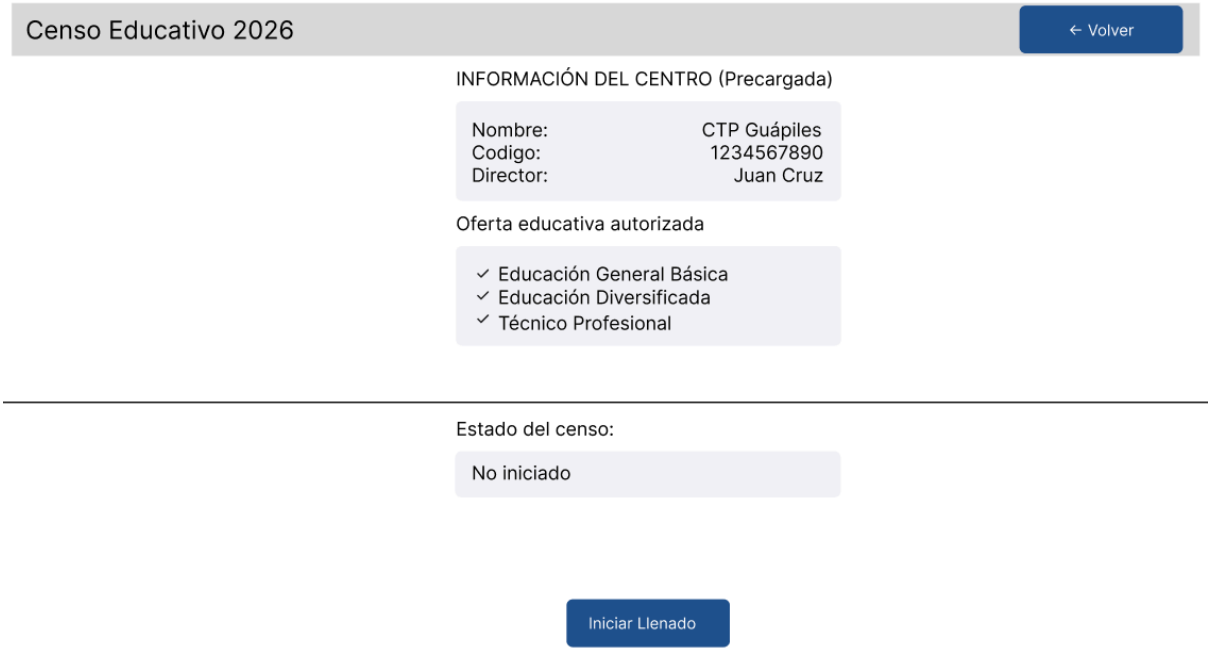
7.2.1.8 RF-08



7.2.1.9 RF-09



7.2.1.10 RF-10

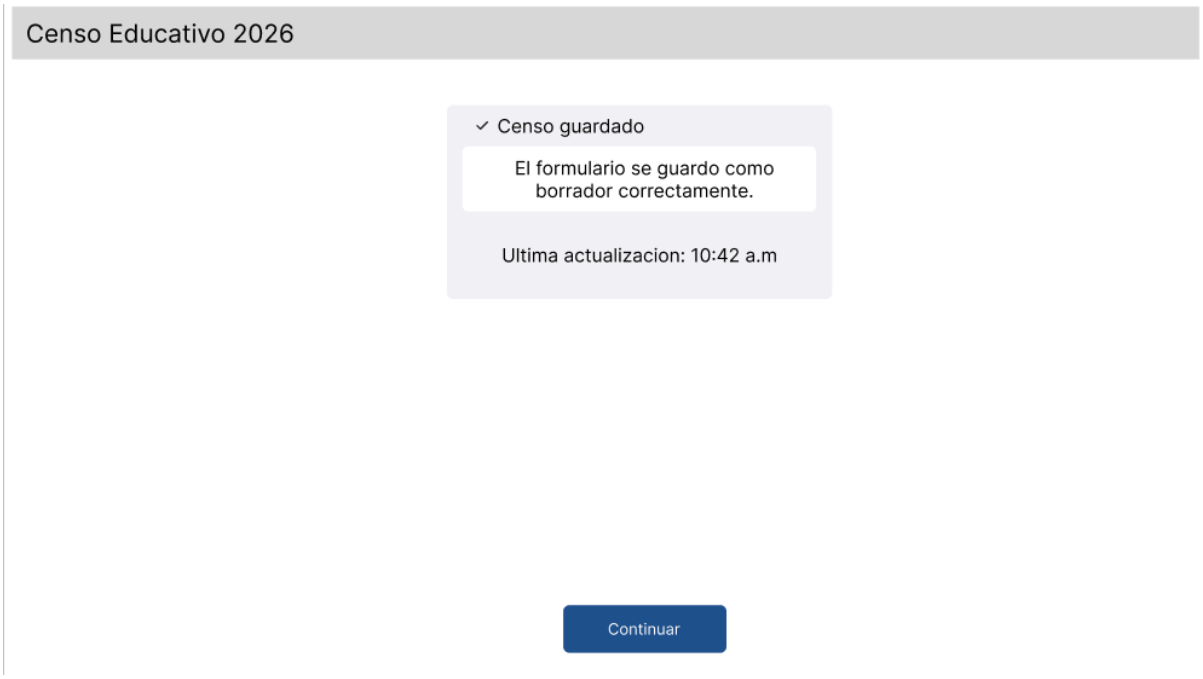


7.2.1.11 RF-11


Censo Educativo 2026
← Volver
INFORMACIÓN DEL CENTRO
Nombre del centro
CTP Guápiles
Código
1234567890
MATRÍCULA
Cantidad de estudiantes
850
Hombres
Mujeres
430
420
← Anterior
Guardar
Siguiente →

7.2.1.12 RF-12

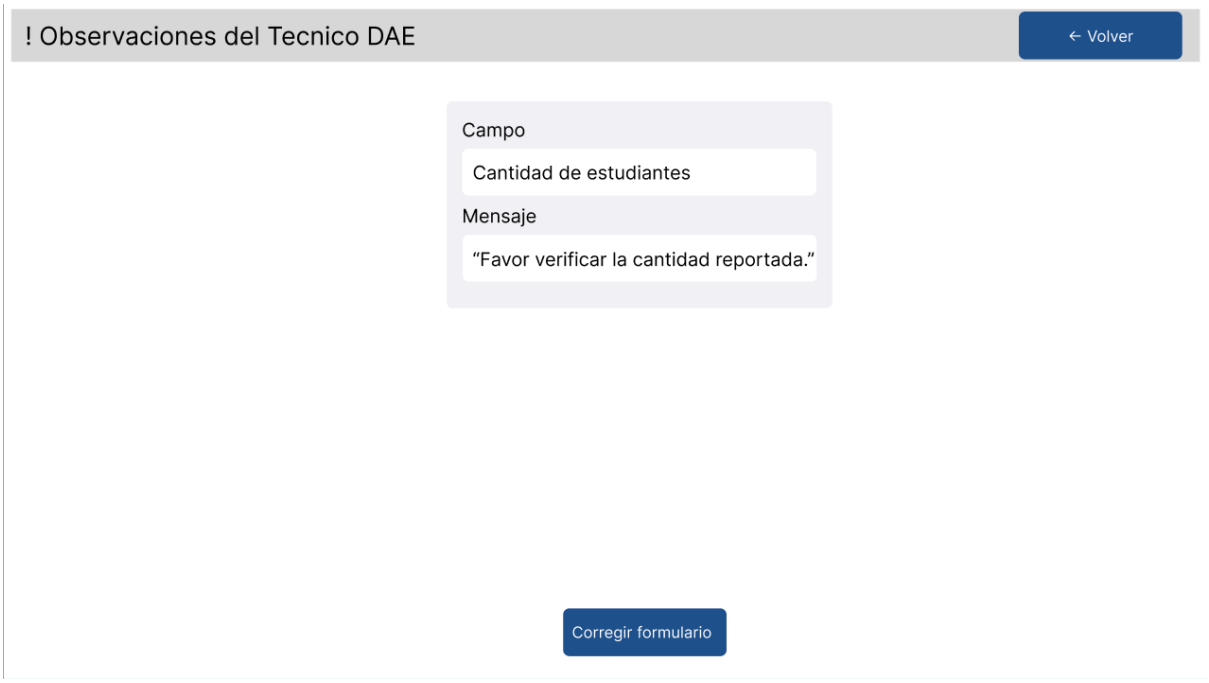

Censo Educativo 2026
← Volver
INFORMACIÓN DEL CENTRO
Nombre del centro
CTP Guápiles
Código
1234567890
MATRÍCULA
Cantidad de estudiantes
850
Hombres
Mujeres
430
420
← Anterior
Guardar borrador
Guardar
Siguiente →

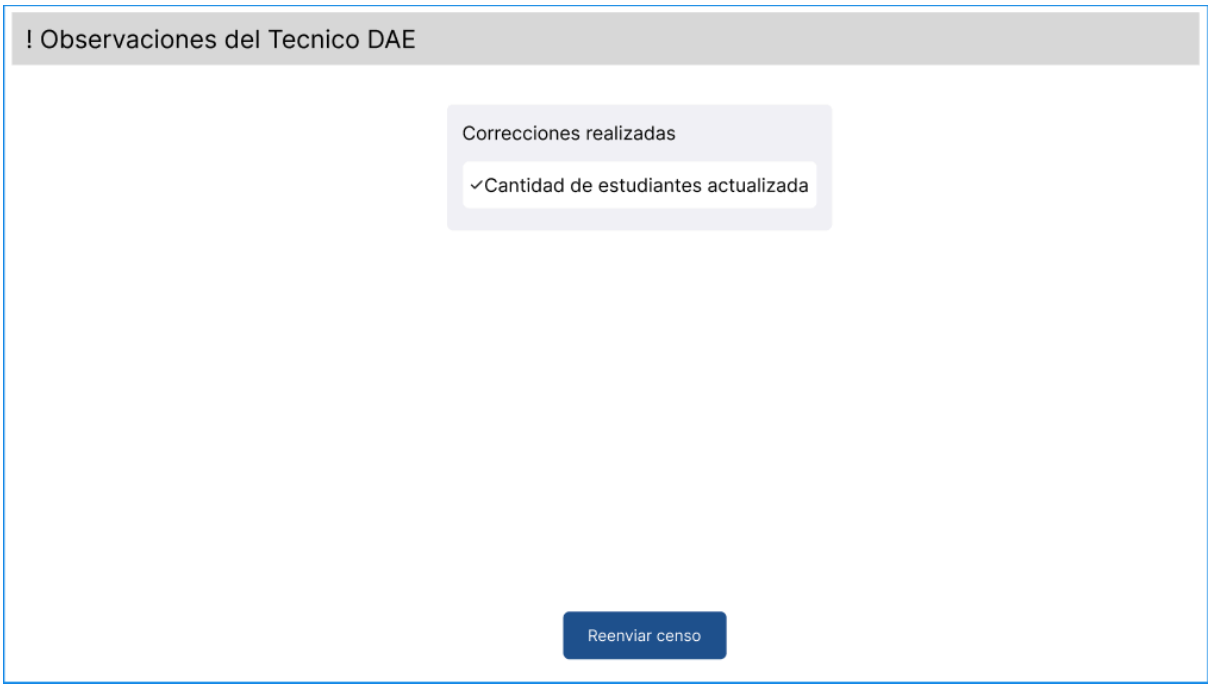


7.2.1.13 RF-13



7.2.1.14 RF-14

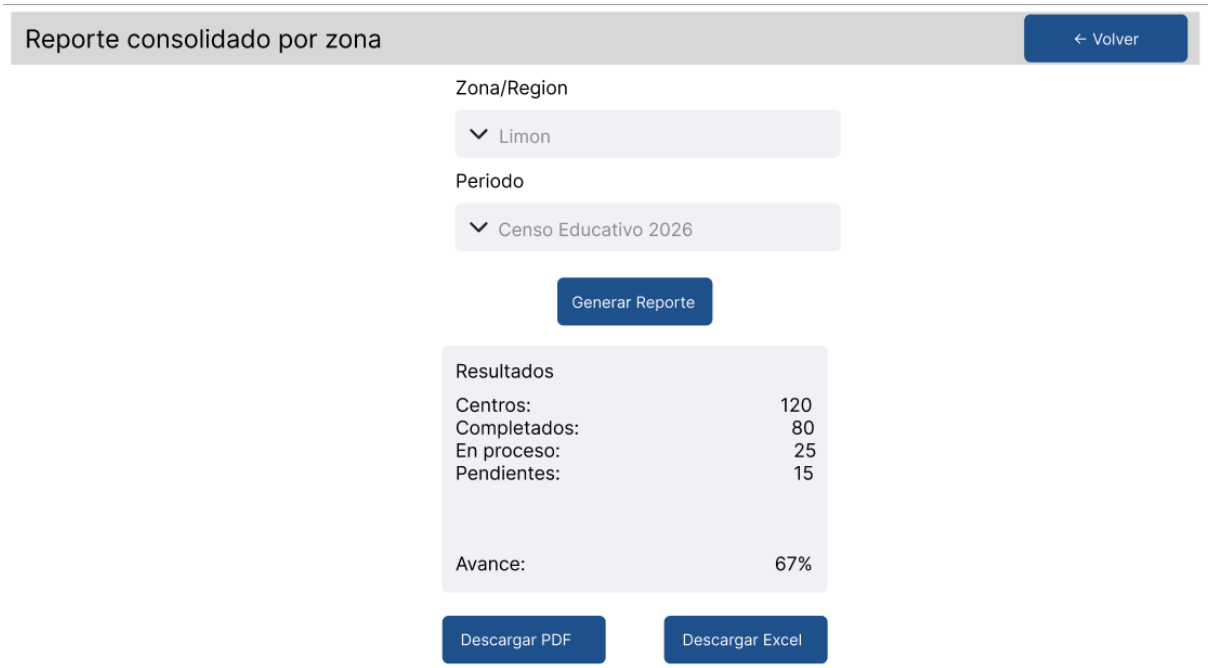

! Observaciones del Tecnico DAE
← Volver
Campo
Cantidad de estudiantes
Mensaje
“Favor verificar la cantidad reportada.”
Corregir formulario


! Observaciones del Tecnico DAE
Correcciones realizadas
✓Cantidad de estudiantes actualizada
Reenviar censo

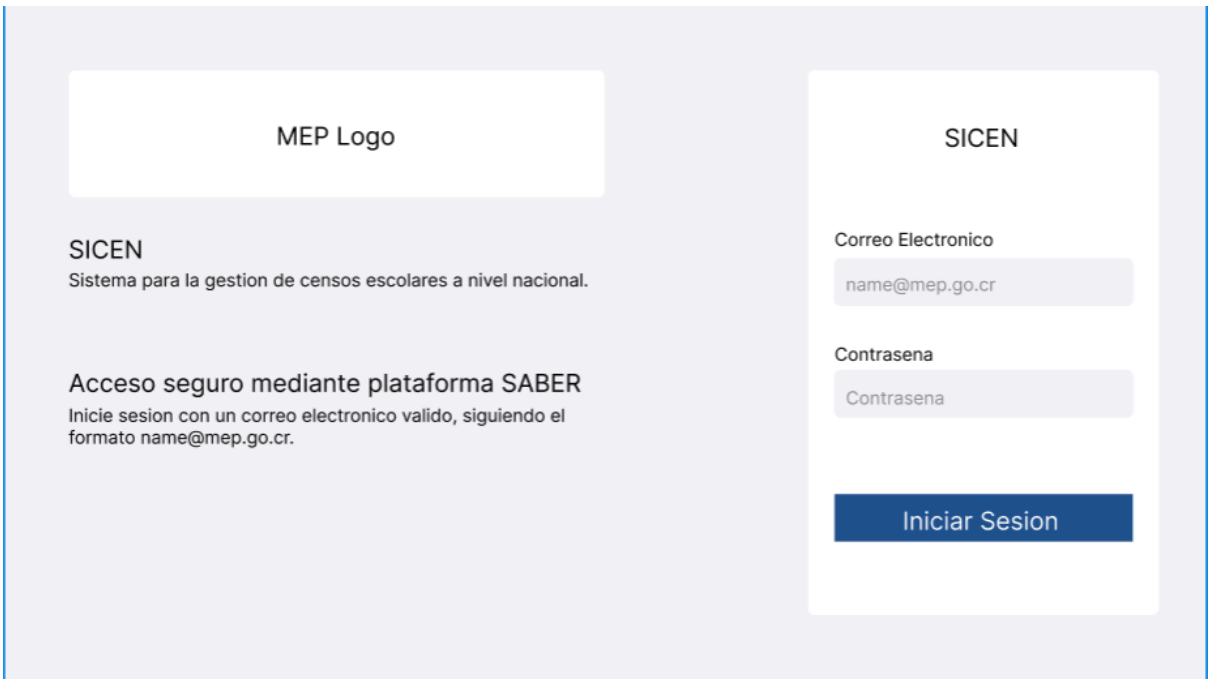
7.2.1.15 RF-15

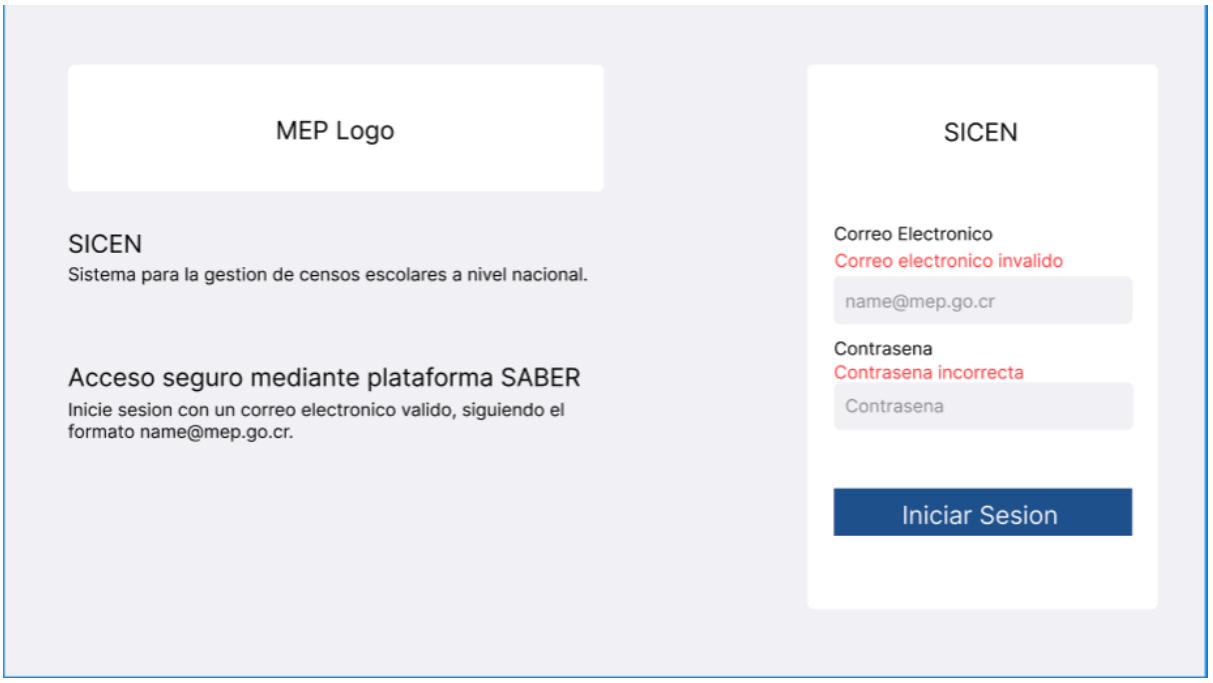


7.2.1.16 RF-16



7.2.1.17 RF-17


MEP Logo
SICEN
Sistema para la gestion de censos escolares a nivel nacional.
Acceso seguro mediante plataforma SABER
Inicie sesion con un correo electronico valido, siguiendo el formato name@mep.go.cr.
SICEN
Correo Electronico
name@mep.go.cr
Contrasena
Contrasena
Iniciar Sesion


MEP Logo
SICEN
Sistema para la gestion de censos escolares a nivel nacional.
Acceso seguro mediante plataforma SABER
Inicie sesion con un correo electronico valido, siguiendo el formato name@mep.go.cr.
SICEN
Correo Electronico
Correo electronico invalido
name@mep.go.cr
Contrasena
Contrasena incorrecta
Contrasena
Iniciar Sesion



7.3 Guía de Estilos

7.3.1 Colores

Paleta institucional basada en azul, con colores semánticos para los estados de los censos.

Uso	Nombre	Hex
Primario (navegación, botones principales, enlaces)	Azul institucional	#1B4F8F
Secundario (acentos, botones secundarios)	Verde azulado	#00796B
Fondo general	Gris muy claro	#F5F7FA
Superficie (tarjetas, formularios)	Blanco	#FFFFFF
Texto principal	Gris oscuro	#1F2933
Texto secundario	Gris medio	#5F6B7A
Éxito	Verde	#2E7D32
Advertencia	Ámbar oscuro	#B45309

Error / peligro	Rojo	#C62828
Información	Azul claro	#0277BD

Colores por estado del censo

Estado	Color
Borrador	Gris #5F6B7A
Enviado	Información #0277BD
Con observaciones	Advertencia #B45309
Aprobado	Éxito #2E7D32
Rechazado	Error #C62828

7.3.2 Tipografía

Familia tipográfica: Inter para títulos y cuerpo, con alternativa system-ui, -apple-system, "Segoe UI", Roboto, sans-serif.

Elemento	Tamaño	Peso
Título de página (h1)	2 rem (32 px)	700
Sección (h2)	1.5 rem (24 px)	600
Subsección (h3)	1.25 rem (20 px)	600
Cuerpo	1 rem (16 px)	400
Etiquetas de formulario	0.875 rem (14 px)	500
Texto de ayuda y notas	0.8125 rem (13 px)	400

Tamaño base de 16 px e interlineado de 1.5.

7.3.3 Componentes

Se utilizan los componentes de Bootstrap 5, personalizados con las siguientes variables CSS:

```
:root {
  --color-primario: #1B4F8F;
  --color-secundario: #00796B;
  --color-fondo: #F5F7FA;
  --color-texto: #1F2933;
  --color-exito: #2E7D32;
  --color-advertencia: #B45309;
  --color-error: #C62828;
  --color-info: #0277BD;
  --radio-borde: 8px;
  --espaciado-base: 8px;
}
```

Componente	Uso en SICEN	Especificación
Barra de navegación / menú lateral	Navegación según el rol del actor	Fondo primario, texto blanco, ítem activo resaltado
Botones	Guardar borrador, Enviar, Aprobar, Rechazar	Primario (acción principal), secundario (contorno) y peligro (Rechazar). Altura mínima de 40 px
Formularios	Formularios censales, registro de colaboradores	Etiqueta sobre el campo, asterisco en obligatorios, mensaje de error debajo en rojo
Tablas	Listado de censos, centros y colaboradores	Encabezado fijo, filas alternadas, paginación y filtros
Tarjetas de indicador	Avance de censos	Número destacado, etiqueta y barra de progreso

Badges de estado	Estado del censo	Colores de la tabla de estados, siempre con texto
Alertas y toasts	Confirmaciones y errores de guardado o envío	Con ícono y texto, cierre manual
Modales de confirmación	Envío definitivo, aprobar o rechazar, cierre de censo	Título claro y dos botones: confirmar y cancelar
Campo de observaciones	Observaciones del Técnico DAE	Área de texto con contador de caracteres
Migas de pan	Ubicación dentro del sistema	Breadcrumb de Bootstrap
Barra de progreso	Avance de censo o región	Con porcentaje visible

El espaciado usa múltiplos de 8 px y los bordes redondeados son de 8 px.

7.4 Restricciones de Usabilidad y Accesibilidad

7.4.1 Usabilidad

- Consistencia: todas las pantallas comparten la misma estructura, componentes y ubicación de acciones principales.
- Navegación por rol: cada actor solo ve las opciones que le corresponden.
- Prevención de errores: las acciones irreversibles (envío definitivo, aprobar, rechazar, cerrar censo) piden confirmación en un modal.
- Validación de formularios: los campos se validan en el momento y los mensajes de error indican con claridad qué corregir.
- Guardado de borrador: el Director puede guardar su avance y retomarlo antes del envío definitivo.
- Retroalimentación: toda acción muestra un resultado visible (éxito, error o carga en proceso).
- Estado visible: el estado del censo se muestra siempre con badge, texto y color.
- Diseño responsivo: las pantallas se adaptan a escritorio, tableta y celular mediante la cuadrícula de Bootstrap.

- Idioma: toda la interfaz y los mensajes están en español (Costa Rica).

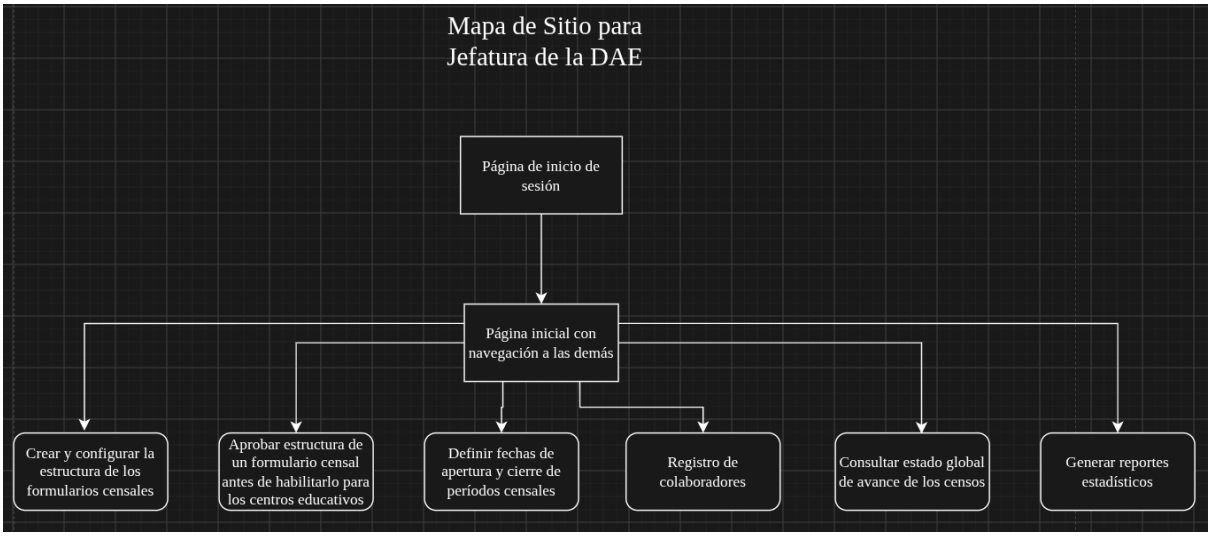
Accesibilidad (referencia: WCAG 2.1, nivel AA)

- Contraste: relación mínima de 4.5:1 en texto normal y 3:1 en texto grande y componentes de interfaz.
- Color: la información nunca se transmite solo con color; se acompaña de texto o ícono.
- Teclado: todas las funciones se pueden usar con teclado, con orden de tabulación lógico y foco visible.
- Formularios: cada campo tiene una etiqueta asociada (label con for) y los errores se asocian al campo.
- Semántica: uso de etiquetas HTML semánticas (header, nav, main, section) y atributos ARIA cuando sea necesario.
- Imágenes e íconos: texto alternativo en imágenes informativas; los íconos decorativos se ocultan a los lectores de pantalla.
- Tamaño de texto: unidades relativas (rem) para que el texto se pueda ampliar hasta un 200 % sin perder contenido.
- Objetivos táctiles: botones y enlaces de al menos 40 px de alto.
- Tablas: encabezados marcados con th y scope.

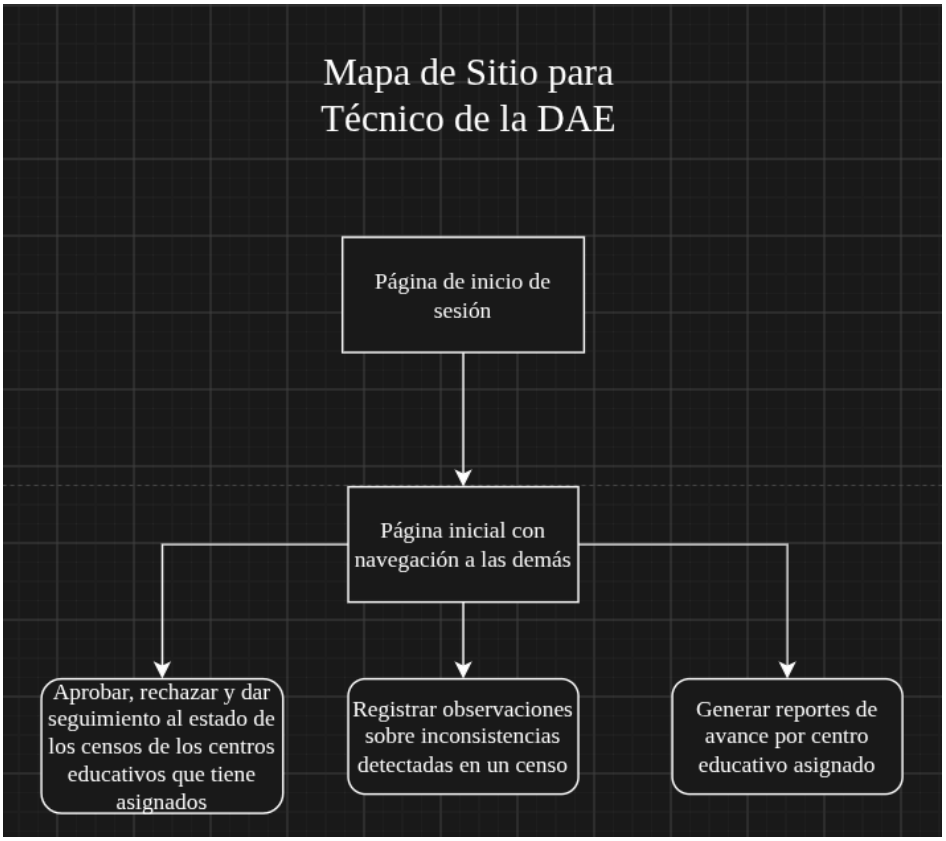
7.5 Diseño de Navegación

7.5.1 Mapas de Sitio

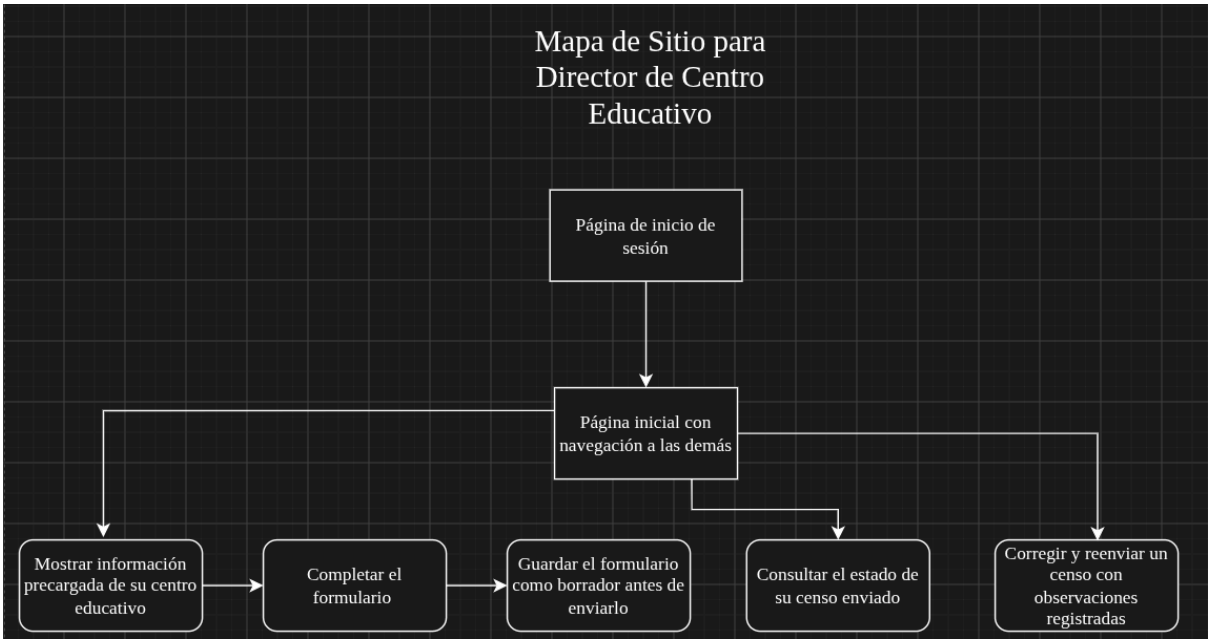
7.5.1.1 Mapa de Sitio para la Jefatura de la DAE



7.5.1.2 Mapa de Sitio para el Técnico de la DAE



7.5.1.3 Mapa de Sitio para el Director del Centro Educativo



7.5.1.4 Mapa de Sitio para el Supervisor

